

Análise do Impacto do Tamanho Médio dos Commits na Manutenibilidade de Sistemas de Software em Python

Eduardo Henrique Moreira dos Santos, Ian dos Reis Novais, João Antonio Nascimento Pires,
João Guilherme Falante Araújo, Mauricio Fernandes Leite, Nalanda Luiza dos Santos,
Roberta Sophia Carvalho Silva

¹Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brazil

Abstract. *Software maintenance is one of the most critical activities in collaborative development, especially in open source projects hosted on platforms such as GitHub. Commits are the fundamental unit of change in versioned systems, and their size may influence key maintenance indicators such as bug introduction, code review complexity, and revert rates. This paper analyzes the association between average commit size and software maintainability in popular Python repositories on GitHub. Using a mining software repositories (MSR) approach, we collected data from 500 repositories filtered by activity, number of contributors, and recency. Statistical analyses show that larger commits are significantly associated with higher bug-introducing and revert rates, while larger pull requests attract more aggregate review comments yet lower comment density per changed line, confirming that smaller, focused changes contribute to more maintainable software.*

Resumo. *A manutenção de software é uma das atividades mais críticas no desenvolvimento colaborativo, especialmente em projetos open source hospedados em plataformas como o GitHub. Os commits representam a unidade fundamental de mudança em sistemas versionados, e seu tamanho pode influenciar indicadores-chave de manutenção, como a introdução de bugs, a complexidade da revisão de código e as taxas de reversão. Este artigo analisa a associação entre o tamanho médio dos commits e a manutenibilidade de software em repositórios Python populares no GitHub. Por meio de uma abordagem de mineração de repositórios de software (MSR), foram coletados dados de 500 repositórios filtrados por atividade, número de contribuintes e recência. As análises estatísticas demonstram que commits maiores estão associados a maiores taxas de introdução de bugs e de reversões; pull requests maiores concentram mais comentários agregados na revisão, porém com menor densidade por linha alterada, confirmando que mudanças menores e focadas contribuem para software com maior manutenibilidade.*

1. Introdução

A manutenção de software responde por uma parcela significativa dos custos do ciclo de vida de um sistema. Em projetos colaborativos hospedados em plataformas como o GitHub, a principal unidade de mudança é o *commit*: um registro atômico das modificações realizadas no código-fonte. Embora amplamente estudada em outros contextos, a relação entre o *tamanho* desses commits e a capacidade de manter o sistema

ao longo do tempo ainda carece de evidências empíricas sistemáticas, especialmente no ecossistema Python.

O problema se torna relevante ao observar que commits excessivamente grandes podem comprometer múltiplos aspectos da qualidade do processo de desenvolvimento: dificultam a revisão de código ao aumentar a carga cognitiva dos revisores, reduzem a rastreabilidade das mudanças, elevam o risco de introdução de defeitos e tornam reversões mais custosas. Diante disso, surge a necessidade de investigar empiricamente como o tamanho dos commits se relaciona com indicadores concretos de manutenção do código.

A motivação deste trabalho origina-se da lacuna existente entre as boas práticas recomendadas pela comunidade de engenharia de software, que frequentemente advogam por commits pequenos e atômicos, e a ausência de evidências quantitativas em larga escala que suportem essas recomendações em projetos Python reais. Compreender essa relação é importante tanto para orientar desenvolvedores e equipes de revisão quanto para fundamentar o desenvolvimento de ferramentas e políticas de qualidade em repositórios open source.

O objetivo geral deste trabalho é analisar a associação entre o tamanho dos commits e seu impacto na manutenibilidade do software, considerando indicadores de manutenção, revisão de código e correção de defeitos, sob a perspectiva de mantenedores, no contexto de projetos open source populares desenvolvidos em Python no GitHub. Como objetivos específicos, busca-se: (i) quantificar a diferença de tamanho entre commits que introduzem bugs (*bug-introducing commits*, BICs) e os demais; (ii) verificar se pull requests maiores geram maior complexidade no processo de revisão de código; e (iii) avaliar como o tamanho dos commits se associa à taxa de reversões e à complexidade ciclomática do código produzido.

Para atingir esses objetivos, o estudo foi organizado em torno de três perguntas de pesquisa derivadas do modelo GQM (*Goal-Question-Metric*):

- **Q1:** Qual a relação entre o tamanho do commit e a ocorrência de bugs?
- **Q2:** O tamanho do pull request influencia a complexidade da revisão de código?
- **Q3:** Qual a relação do tamanho dos commits com a manutenibilidade do código?

O restante deste artigo está organizado da seguinte forma. A Seção 2 apresenta os trabalhos relacionados. A Seção 3 descreve a metodologia adotada. A Seção 4 discute os resultados obtidos para cada pergunta de pesquisa. Por fim, a Seção 5 apresenta as conclusões e trabalhos futuros.

2. Trabalhos Relacionados

Hattori e Lanza [Hattori and Lanza 2008] realizaram um estudo sobre a natureza dos commits em projetos open source, caracterizando-os quanto ao tipo de mudança (correção de bugs, refatoração, implementação de funcionalidades e manutenção). Os autores propuseram uma classificação de commits em pequenos, médios e grandes com base no número de arquivos e linhas modificadas, que é adotada como base para a categorização utilizada no presente trabalho.

Arafat e Riehle [Arafat and Riehle 2009] analisaram a distribuição do tamanho de commits em projetos open source e observaram que a maioria dos commits tende a ser

pequena, com poucos arquivos e poucas linhas alteradas. O estudo evidencia que grandes commits são raros, mas potencialmente mais impactantes para a qualidade do software, motivando investigações mais aprofundadas sobre seus efeitos na manutenção.

Śliwerski, Zimmermann e Zeller [Śliwerski et al. 2005] introduziram o algoritmo SZZ para identificar commits que introduzem bugs (*bug-introducing commits*), rastreando o histórico de modificações via `git blame` a partir de commits de correção. Esse trabalho é fundamental para o presente estudo, pois a identificação dos BICs adota uma variante moderna do SZZ (MA-SZZ) para relacionar o tamanho dos commits à introdução de defeitos.

Kalliamvakou et al. [Kalliamvakou et al. 2014] investigaram as promessas e os perigos da mineração de repositórios no GitHub, alertando para vieses como repositórios pessoais, projetos inativos e repositórios de fins educacionais. Esse estudo fundamenta as decisões de curadoria adotadas neste trabalho, em especial os critérios de filtragem por número de contribuintes, período de atividade e quantidade mínima de commits.

Munaiah et al. [Munaiah et al. 2017] propuseram diretrizes para curadoria de projetos de engenharia de software a partir do GitHub, distinguindo repositórios de software real de repositórios de experimentos, tutoriais e projetos acadêmicos. A metodologia do presente trabalho segue essas diretrizes ao priorizar repositórios com múltiplos contribuintes, longa história e atividade recente.

Gousios, Storey e Bacchelli [Gousios et al. 2016] estudaram o fluxo de trabalho baseado em pull requests a partir da perspectiva dos contribuintes, identificando fatores que facilitam ou dificultam a aceitação de contribuições. Os autores apontam que o tamanho da pull request é um fator relevante no processo de revisão, o que se relaciona diretamente com a segunda pergunta de pesquisa deste trabalho.

Mamun, Berger e Hansson [Mamun et al. 2017] realizaram um estudo empírico sobre correlações entre métricas de código-fonte, incluindo linhas de código, complexidade ciclomática e taxa de defeitos. O trabalho apoia a escolha das métricas utilizadas neste estudo, em especial a combinação de LOC e complexidade ciclomática como indicadores de manutenibilidade.

3. Metodologia

3.1. Visão Geral

Este trabalho adota uma abordagem de Mineração de Repositórios de Software (MSR), conduzida em sete etapas: (A) seleção dos repositórios; (B) coleta de dados via GitHub API; (C) normalização dos dados; (D) filtragem e seleção; (E) cálculo de métricas; (F) análise estatística; e (G) análise e interpretação dos resultados. A Figura 1 ilustra esse fluxo.

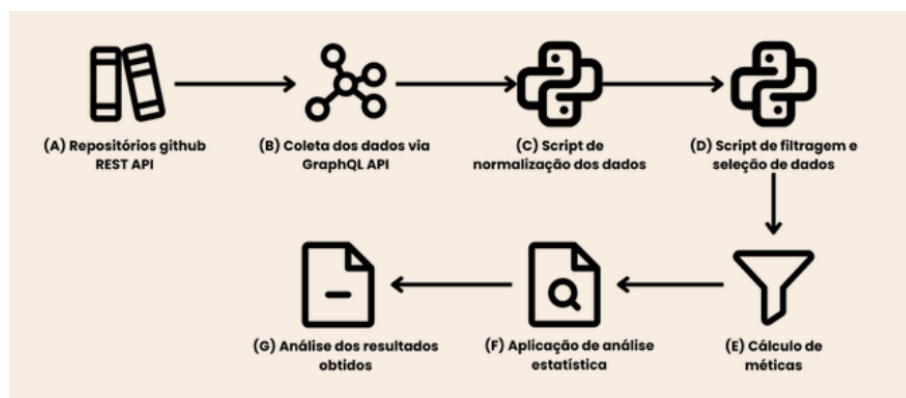


Figura 1. Visão geral da metodologia adotada no estudo.

3.2. Seleção dos Repositórios

Foram selecionados até 500 repositórios Python no GitHub, obedecendo aos seguintes critérios de inclusão: (i) Python como linguagem principal; (ii) janela temporal de 5 anos; (iii) mais de 4 contribuintes; (iv) pelo menos 100 commits; (v) pelo menos um commit no último ano; (vi) apenas commits com no máximo 1 ano de idade foram validados nas análises; e (vii) priorização por popularidade (número de estrelas). Os critérios garantem que os repositórios selecionados representem projetos ativos e colaborativos reais [Kalliamvakou et al. 2014, Munaiah et al. 2017].

3.3. Coleta e Processamento dos Dados

Os dados de commits, pull requests e issues foram extraídos por meio da GitHub REST API e da GraphQL API. Em seguida, scripts Python foram utilizados para normalização, limpeza e padronização dos dados coletados. Uma etapa de filtragem posterior removeu registros inconsistentes e selecionou apenas os dados relevantes para cada pergunta de pesquisa.

3.4. Definição das Perguntas de Pesquisa e Métricas

As perguntas de pesquisa foram definidas segundo o modelo GQM:

3.4.1. Q1 – Relação entre tamanho do commit e ocorrência de bugs

Hipótese nula (H0): Não existe associação entre a classe de tamanho do commit e a probabilidade de ele ser bug-introducing, isto é, a taxa de BICs é equivalente entre as classes.

Hipótese alternativa (H1): A probabilidade de um commit ser bug-introducing cresce com a classe de tamanho; commits maiores apresentam taxa de BICs superior à dos menores.

Variável independente: Classe de tamanho do commit (pequeno, médio ou grande), definida pelos limiares de Hattori e Lanza (2008) sobre o número de arquivos modificados e as LOC alteradas.

Variável dependente: Classificação do commit como *bug-introducing* (BIC), identificado pelo algoritmo MA-SZZ a partir dos commits de correção via `git blame`.

Métricas:

- M1.1: LOC modificadas e número de arquivos alterados por commit;
- M1.2: Classe de tamanho do commit (pequeno / médio / grande), conforme Hattori e Lanza (2008);
- M1.3: Classificação binária como BIC (MA-SZZ);
- M1.4: Taxa de BICs por classe e por repositório ($n_BIC/total_commits$).

Métodos: Teste de tendência de Cochran-Armitage (taxa de BICs ao longo das classes ordinais de tamanho), teste qui-quadrado de associação com Cramer's V (composição das classes entre BICs e não-BICs) e correlação de Spearman ρ entre a proporção de commits grandes e a taxa de BICs por repositório.

3.4.2. Q2 – Influência do tamanho do PR na complexidade da revisão

Hipótese nula (H0): Não existe associação entre o tamanho do pull request e os indicadores de complexidade da revisão (volume e densidade de comentários, latência até a primeira revisão formal ou duração do ciclo até o fechamento).

Hipótese alternativa (H1): Pull requests maiores, com mais linhas e mais arquivos alterados, tendem a gerar mais comentários na revisão, a aumentar o tempo até a primeira revisão formal e a prolongar o ciclo até o fechamento.

Variável independente: LOC por PR (linhas adicionadas + removidas), número de arquivos alterados por PR e categoria de tamanho (pequeno, médio e grande), conforme [Hattori and Lanza 2008], adaptada a PRs: pequeno ($LOC \leq 25$ e arquivos ≤ 5), grande ($LOC \geq 125$ ou arquivos > 5), médio (demais casos).

Variáveis dependentes: Volume de comentários por PR (`comments + review_comments`), densidade de comentários por LOC (M2.3'), tempo até a primeira revisão formal (`first_review_at - created_at`, em horas) e tempo até o fechamento (`closed_at - created_at`, em horas, PRs fechadas).

Subconjuntos analíticos: S3 — PRs com `first_review_at` válido (M2.4); S4 — PRs fechadas com intervalo de fechamento entre 0 e 8.760 h (M2.5).

Métricas:

- M2.1: LOC por PR;
- M2.2: Número de arquivos alterados por PR;
- M2.3: Total de comentários por PR;
- M2.3': Comentários totais / LOC (densidade de discussão por linha);
- M2.4: Tempo até a primeira revisão formal (horas, S3);
- M2.5: Tempo até o fechamento (horas, S4).

Métodos: Correlação de Spearman com $\log_1 p(LOC)$ e $\log_1 p(horas)$ para reduzir a assimetria das distribuições; teste Mann-Whitney U comparando PRs pequenas e grandes nos cortes $LOC \leq 25$ vs. > 125 (M2.3 e M2.4: H1 grande $>$ pequeno; M2.3': H1 pequeno $>$ grande na densidade) e arquivos ≤ 5 vs. > 5 .

3.4.3. Q3 – Relação entre tamanho dos commits e manutenibilidade

Hipótese nula (H0): Não existe diferença na taxa de reversões nem na complexidade ciclomática entre commits de diferentes tamanhos.

Hipótese alternativa (H1): Commits menores estão associados a um ciclo de manutenção mais ágil (maior proporção de bug fixes, menor taxa de reverts), enquanto commits grandes apresentam maior incidência de reversões devido à dificuldade de validação completa antes do merge.

Variável independente: Tamanho do commit, categorizado em pequeno, médio e grande conforme [Hattori and Lanza 2008].

Variáveis dependentes: Taxa de reverts/rollbacks por categoria, complexidade ciclomática máxima (CC máx.) e densidade de complexidade (CC/LOC).

Métricas:

- M3.1: LOC médio por commit;
- M3.2: Complexidade ciclomática / LOC (CC/LOC);
- M3.3: Taxa de reverts/rollbacks.

Métodos: Comparação descritiva e inferencial entre os três grupos de tamanho de commit, com cálculo de CC máx. via análise da árvore sintática da linguagem e identificação de reverts por NLP sobre as mensagens de commit.

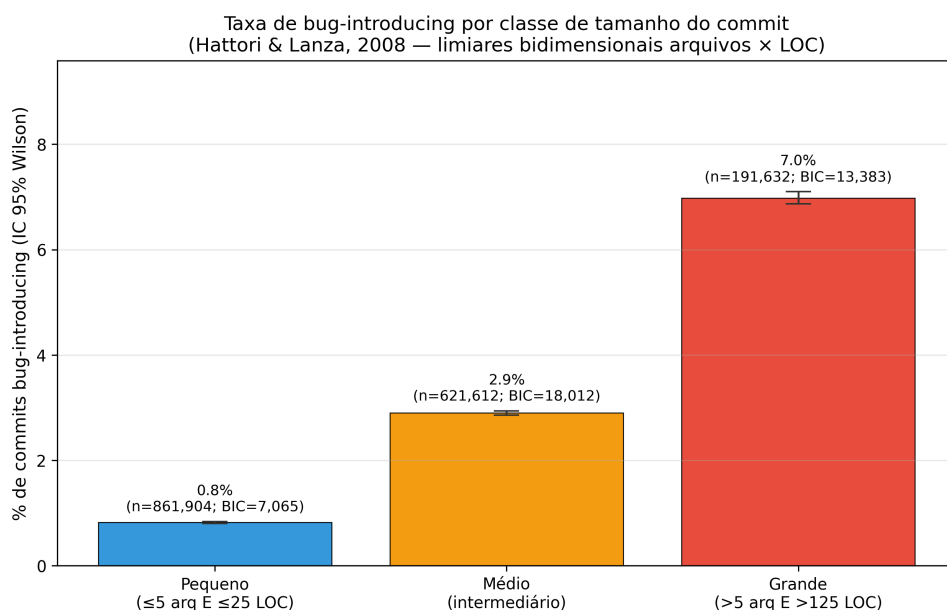
4. Resultados

4.1. Q1 – Relação entre tamanho do commit e ocorrência de bugs

A análise considerou 1.675.148 commits Python (cinco repositórios foram removidos por incompatibilidades técnicas ou erros recorrentes de execução: `pytorch/pytorch`, que falhou por incompatibilidade de filesystem NTFS *case-insensitive*, `Significant-Gravitas/AutoGPT`, `Genesis-Embodied-AI/genesis-world`, `ccxt/ccxt` e `ray-project/ray`). Do total, 38.460 commits (2,3%) foram classificados como BICs pelo algoritmo MA-SZZ e 1.636.688 como commits não bug-introducing. Cada commit foi enquadrado em uma das três classes de tamanho de Hattori e Lanza (2008): **pequeno** (≤ 5 arquivos e ≤ 25 LOC), **grande** (> 5 arquivos e > 125 LOC) e **médio** (demais casos).

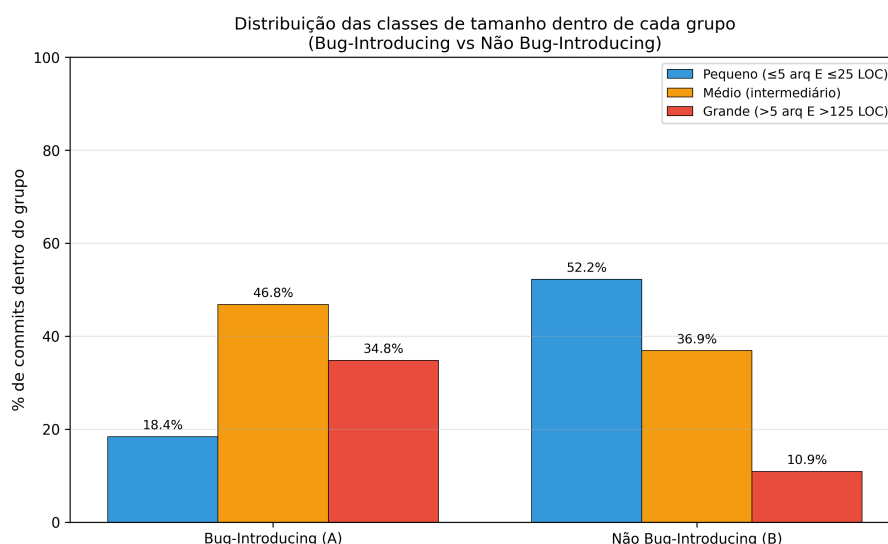
A taxa de BICs cresce de forma monotônica com o tamanho do commit (Figura 2): 0,82% (IC 95% Wilson: 0,80–0,84; $n = 861.904$) nos pequenos, 2,90% (2,86–2,94; $n = 621.612$) nos médios e 6,98% (6,87–7,10; $n = 191.632$) nos grandes, uma razão de aproximadamente 8,5 vezes entre as classes extremas. O teste de tendência de Cochran-Armitage confirma esse crescimento como estatisticamente significativo ($Z = 163,54$; $p < 10^{-50}$), rejeitando a hipótese nula com ampla margem.

Figura 2. Taxa de *bug-introducing commits* por classe de tamanho (Hattori e Lanza, 2008).



A composição das classes difere de forma marcante entre os grupos (Figura 3): entre os BICs, 34,8% são commits grandes e apenas 18,4% são pequenos, enquanto entre os não-BICs a proporção se inverte (10,9% grandes contra 52,2% pequenos). A associação é significativa pelo teste qui-quadrado ($\chi^2 = 28.149$; gl = 2; $p < 10^{-50}$), com tamanho de efeito pequeno em magnitude, porém de direção clara (Cramer's V = 0,13).

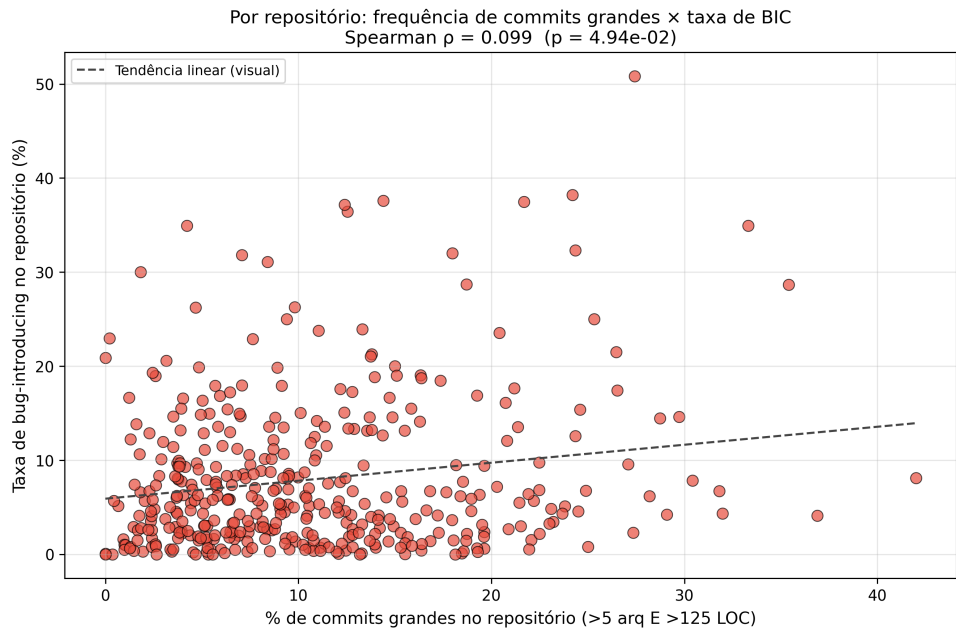
Figura 3. Distribuição das classes de tamanho dentro de cada grupo (BIC vs. não-BIC).



A correlação de Spearman entre a proporção de commits grandes por repositório e a taxa de BICs foi positiva e estatisticamente significativa, porém **fraca**: $\rho = 0,10$ ($p = 0,049$;) (Figura 4). Repositórios com maior proporção de commits grandes tendem a apresentar taxa de BICs ligeiramente maior, mas esse efeito no nível de projeto é muito

mais sutil que o observado no nível do commit.

Figura 4. Correlação por repositório entre a proporção de commits grandes e a taxa de BICs.



Hipótese comprovada: commits maiores têm probabilidade substancialmente maior de introduzir bugs no nível individual, a taxa de BICs é cerca de 8,5 vezes maior nos commits grandes do que nos pequenos, com tendência e associação estatisticamente significativas. O mesmo padrão se manifesta entre projetos, ainda que de forma bem mais tênue ($\rho = 0,10$), indicando que o sinal é predominantemente do nível do **commit**, e não do **projeto**.

4.2. Q2 – Influência do tamanho do PR na complexidade da revisão

A análise considerou 69.615 pull requests de 434 repositórios Python populares no GitHub, classificados em pequeno, médio e grande conforme Hattori e Lanza [Hattori and Lanza 2008]: 21.856, 14.603 e 33.156 PRs, respectivamente. Para a latência até a primeira revisão formal (M2.4), utilizou-se o subconjunto S3 — 32.651 PRs (46,9% da base) com `first_review_at` válido na API; para o tempo até o fechamento (M2.5), o subconjunto S4 — 58.448 PRs fechadas com tempo de ciclo válido (Seção 3). A Tabela 1 resume os principais indicadores por categoria Hattori; M2.4 e M2.5 referem-se a S3 e S4, respectivamente.

Tabela 1. Indicadores de revisão por categoria de tamanho de PR (Hattori). M2.4 e M2.5 calculados em S3 e S4, respectivamente.

Tamanho	Qtd.	M2.3 média (coment.)	M2.3' med. (com/LOC)	M2.4 med. (S3, min)	M2.5 med. (S4, h)
Pequeno	21.856	1,58	0,125	3,7	4,04
Médio	14.603	2,45	0,024	4,7	10,19
Grande	33.156	5,67	0,004	5,6	10,92

Quanto ao **volume de comentários** (M2.3), a associação com o tamanho é positiva e estatisticamente significativa: a correlação de Spearman entre $\log_1 p(\text{LOC})$ e o total de comentários foi $\rho = 0,21$ ($p < 0,001$) na base completa e $\rho = 0,42$ ($p < 0,001$) no subconjunto S3. No teste de Mann-Whitney U ($\text{LOC} \leq 25$ vs. > 125), PRs grandes apresentam em média cerca de 3,8 vezes mais comentários que as pequenas (1,58 vs. 5,94).

Quanto à **densidade de comentários por LOC** (M2.3'), o comportamento é inversamente proporcional ao tamanho, conforme ilustrado na Figura 5. PRs pequenas concentram mediana de 0,125 comentários por linha alterada, contra 0,024 (médios) e 0,004 (grandes), padrão análogo à queda de CC/LOC na Subseção 4.3, sugerindo revisão proporcionalmente mais superficial em mudanças extensas. A correlação de Spearman entre $\log_1 p(\text{LOC})$ e comentários por LOC foi $\rho = -0,36$ ($p < 0,001$) na base com $\text{LOC} > 0$ e $\rho = -0,50$ ($p < 0,001$) em S3.

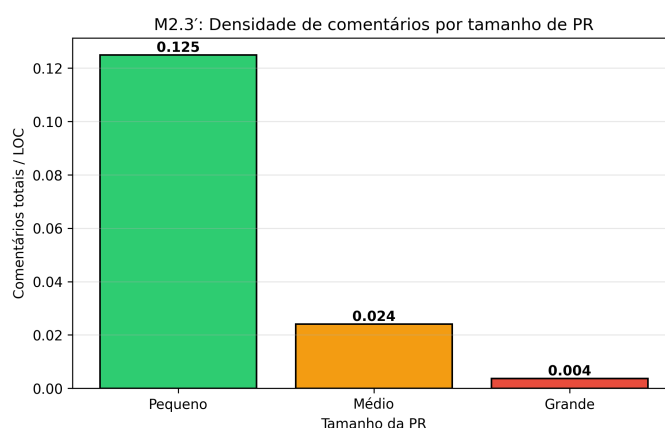


Figura 5. Densidade de comentários por LOC (M2.3') por categoria de tamanho de PR.

Para o tempo até a **primeira revisão formal** (M2.4), embora a diferença seja estatisticamente significativa ($p < 0,001$), o coeficiente de Spearman permanece baixo ($\rho \approx 0,11$ em S3), com medianas de 3,6 e 5,7 minutos entre PRs pequenas e grandes no corte LOC (Mann-Whitney; cf. 3,7–5,6 min por categoria na Tabela 1), efeito detectável, porém de magnitude prática limitada. Fatores como disponibilidade da equipe e carga de trabalho parecem ter maior peso na latência inicial.

O **tempo até o fechamento** (M2.5), ilustrado na Figura 6, cresce monotonicamente entre as faixas de LOC no subconjunto S4: 3,95 h ($\text{LOC} \leq 25$), 8,44 h ($\text{LOC} 26\text{--}125$) e 12,70 h ($\text{LOC} > 125$) na mediana, cerca de 3,2 vezes entre os extremos. Esse contraste por corte de LOC é mais acentuado do que a progressão por categoria Hattori na tabela (4,04 h a 10,92 h), evidenciando que o tamanho pesa mais no ciclo de integração do que no instante inicial da revisão formal.

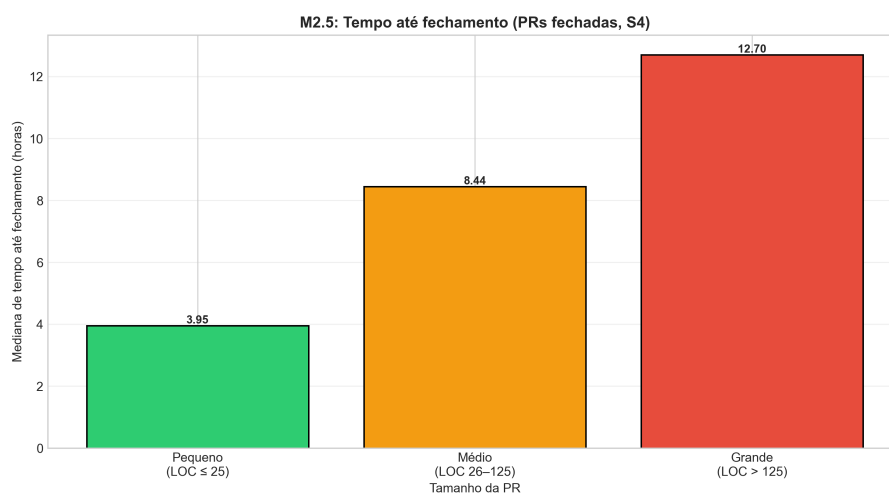


Figura 6. Mediana de tempo até fechamento (M2.5) por faixa de LOC (S4): pequeno (≤ 25), médio (26–125) e grande (> 125).

Hipótese comprovada de forma parcial e qualificada: o tamanho do PR influencia a complexidade da revisão no **volume de comentários** (M2.3) e na **duração do ciclo** até o fechamento (M2.5), enquanto a **densidade de comentários por LOC** (M2.3') decresce com o tamanho, ou seja, PRs maiores recebem menos discussão relativa ao LOC alterado, em linha com a diluição de CC/LOC na Subseção 4.3. A latência até a primeira revisão formal (M2.4) é estatisticamente associada ao tamanho, porém de magnitude prática limitada na mediana típica.

4.3. Q3 – Relação entre tamanho dos commits e manutenibilidade

A análise foi realizada sobre 560.667 commits classificados em três categorias conforme [Hattori and Lanza 2008]: 254.663 pequenos, 228.527 médios e 77.477 grandes. A Tabela 2 resume os principais indicadores por categoria.

Tabela 2. Indicadores de manutenibilidade por categoria de tamanho de commit

Tamanho	Qtd.	LOC médio (M3.1)	CC/LOC (M3.2)	Taxa revert (M3.3)	Taxa bug fix
Pequeno	254.663	7,8	1,96	1,26%	36,18%
Médio	228.527	285,8	0,16	1,99%	35,15%
Grande	77.477	7.512,4	0,04	3,56%	31,15%

Quanto à densidade de complexidade ciclomática (CC/LOC), o comportamento é inversamente proporcional ao tamanho, conforme ilustrado na Figura 7. Commits pequenos apresentam alta densidade lógica (CC/LOC = 1,96), indicando que as modificações são cirúrgicas e focadas, um patch direto no problema. À medida que o tamanho cresce, a densidade cai acentuadamente: 0,16 para commits médios e apenas 0,04 para commits grandes. Isso sugere que commits grandes introduzem código com lógica mais diluída em um volume elevado de linhas, tornando a revisão e os testes mais custosos.

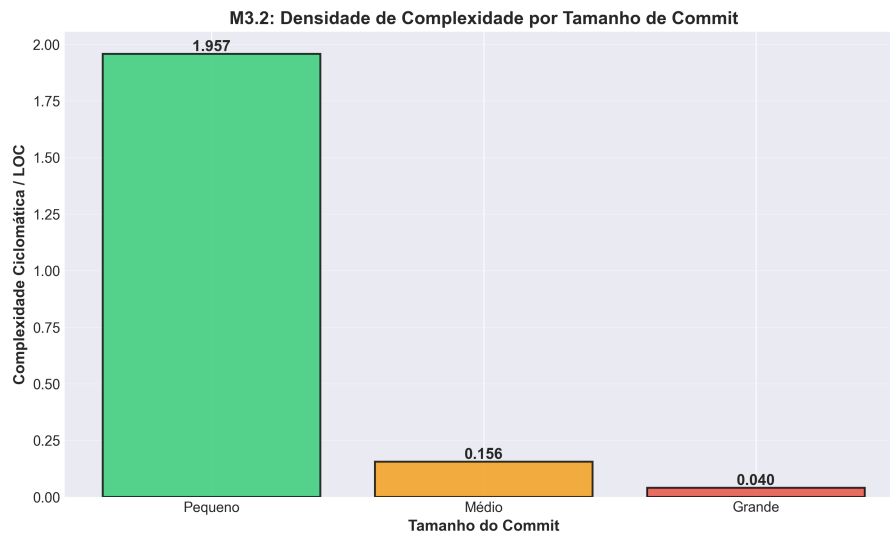


Figura 7. Densidade de complexidade ciclomática (CC/LOC) por categoria de tamanho de commit.

A taxa de reversões cresce monotonamente com o tamanho do commit: de 1,26% nos commits pequenos para 1,99% nos médios e 3,56% nos grandes, um aumento de aproximadamente 2,8 vezes entre os extremos. Esse padrão, ilustrado na Figura 8, é consistente com a hipótese de que commits grandes dificultam a validação completa antes do merge, elevando a incidência de efeitos colaterais e, consequentemente, a necessidade de reversão.

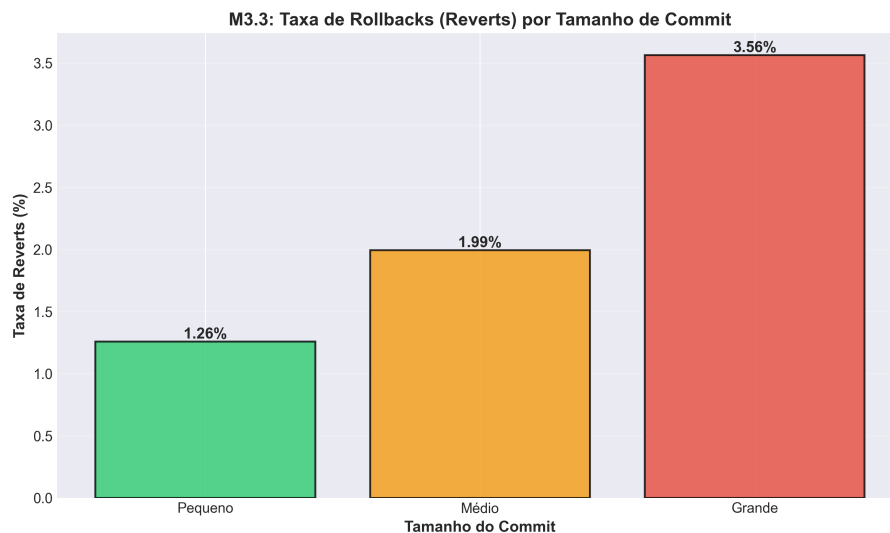


Figura 8. Taxa de rollbacks (reverts) por categoria de tamanho de commit.

A taxa de bug fixes também decresce com o tamanho (36,18% → 35,15% → 31,15%), reforçando que commits menores estão mais associados a ações corretivas precisas e atômicas, características desejáveis em um ciclo de manutenção ágil.

Hipótese comprovada: O tamanho do commit tem relação direta e inversamente proporcional à manutenibilidade. Commits menores facilitam correções atômicas

e rápidas (maior taxa de bug fixes, menor taxa de revert e maior densidade ciclomática), enquanto commits grandes concentram maior carga cognitiva no code review, maior incidência de efeitos colaterais e maior taxa de código revertido.

5. Conclusão

Na **Q1**, observou-se que commits classificados como introdutores de bugs (BICs) são consistentemente maiores que os demais, apresentando diferenças expressivas tanto do ponto de vista estatístico quanto prático (mediana de 143 vs. 23 LOC; Cliff's Delta 0,48). Esse resultado sugere que mudanças extensas tendem a aumentar a carga cognitiva necessária para compreender, validar e testar o código alterado, elevando a probabilidade de que defeitos sejam introduzidos durante o processo de desenvolvimento. Dessa forma, os achados reforçam a importância de promover alterações menores e mais focadas como estratégia para reduzir riscos de qualidade.

Na **Q2**, verificou-se que *pull requests* maiores demandam maior esforço de revisão, refletido no aumento do volume de comentários e no prolongamento do ciclo até o fechamento. Entretanto, a densidade de comentários por linha alterada diminui à medida que o tamanho da mudança cresce, indicando que revisões de alterações extensas tendem a ser proporcionalmente menos detalhadas. Esse comportamento sugere que o aumento do escopo das mudanças pode dificultar a análise criteriosa do código, reduzindo a capacidade da revisão em identificar problemas potenciais antes da integração.

Na **Q3**, observou-se que commits maiores apresentam taxas mais elevadas de reversão e padrões associados a menor manutenibilidade. Esses resultados indicam que mudanças extensas são mais difíceis de validar e integrar adequadamente, aumentando a probabilidade de efeitos colaterais e correções posteriores. Em contrapartida, commits menores tendem a favorecer intervenções mais localizadas e controláveis, contribuindo para ciclos de manutenção mais eficientes e previsíveis.

Em conjunto, os resultados convergem para uma mesma evidência: o aumento do tamanho das mudanças está associado a maiores desafios de qualidade, revisão e manutenção. Assim, o estudo fornece evidências empíricas de que a adoção de commits e *pull requests* menores e mais focados pode contribuir para reduzir a introdução de defeitos, favorecer revisões mais efetivas e tornar o processo de manutenção de software mais ágil e sustentável ao longo do tempo.

Referências

- Arafat, O. and Riehle, D. (2009). The commit size distribution of open source software. In *42nd Hawaii International Conference on System Sciences (HICSS)*, pages 1–8, Waikoloa, HI, USA.
- Gousios, G., Storey, M.-A., and Bacchelli, A. (2016). Work practices and challenges in pull-based development: The contributor's perspective. In *38th International Conference on Software Engineering (ICSE)*, pages 285–296, Austin, TX, USA. ACM.
- Hattori, L. P. and Lanza, M. (2008). On the nature of commits. In *23rd IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, pages 63–71, L'Aquila, Italy.

- Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2014). The promises and perils of mining GitHub. In *11th Working Conference on Mining Software Repositories (MSR)*, pages 92–101, Hyderabad, India. ACM.
- Mamun, M. A. A., Berger, C., and Hansson, J. (2017). Correlations of software code metrics: An empirical study. In *27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement (IWSM Mensura)*, pages 255–266, New York, NY, USA. ACM.
- Munaiah, N., Kroh, S., Cabrey, C., and Nagappan, M. (2017). Curating GitHub for engineered software projects. *Empirical Software Engineering*, 22(6):3219–3253.
- Śliwerski, J., Zimmermann, T., and Zeller, A. (2005). When do changes induce fixes? In *International Workshop on Mining Software Repositories (MSR)*, Saint Louis, Missouri, USA. ACM.